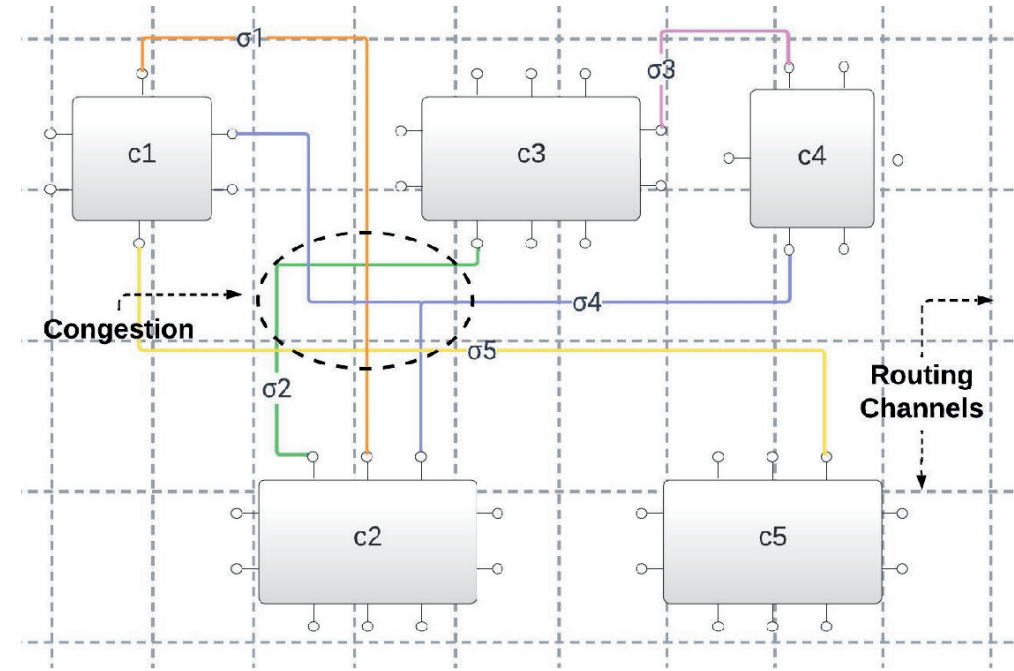
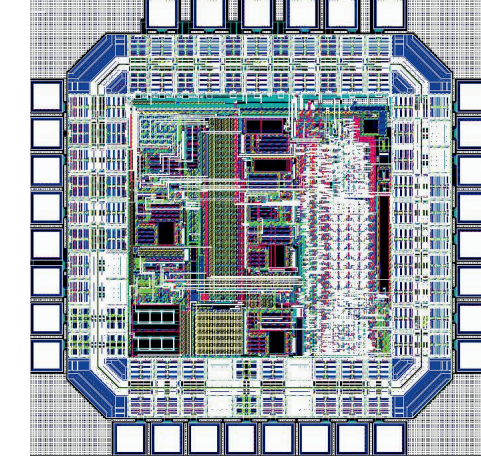




Introduction

Background

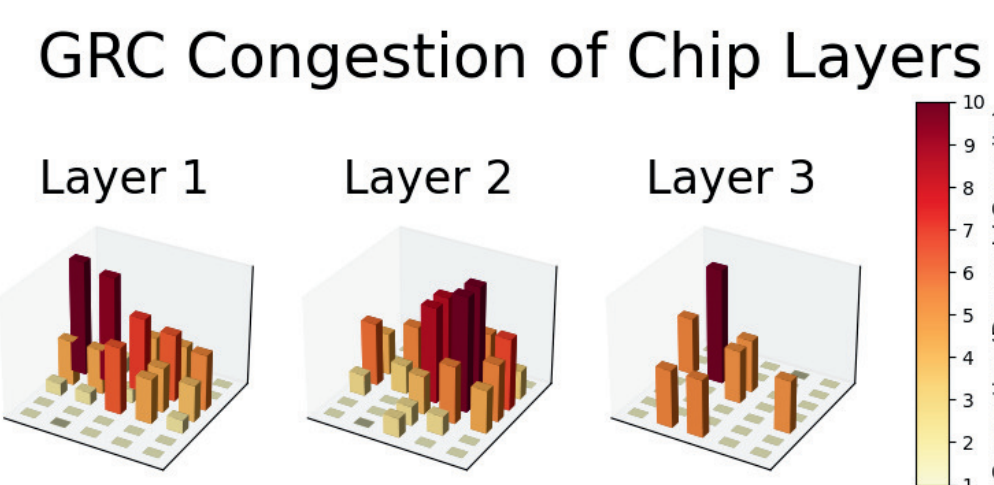
- Chip design** has increased exponentially in terms of complexity, leading to significant challenges in predicting and reducing wire **congestion**.
- Congestion** describes areas with **excess wire routing density** and power **demand**, which **negatively impacts performance** through overheating and signal interference.



- Many **SOTA** Graph ML techniques are **less effective** at dealing with **chip layouts** due to their connectivity structure.
- The existing **DE-HNN (Luo, 2024)** model aims to **address** the lack of **generalizability** in **chip graph representations**.

Motivation

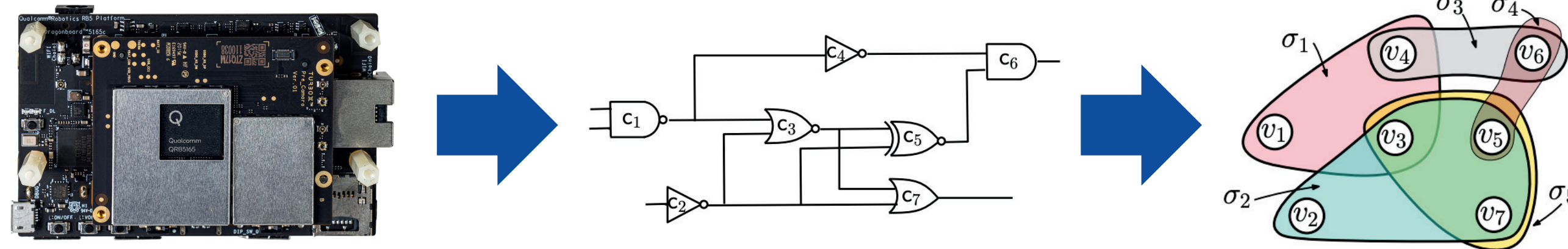
- Predicting and mitigating **congestion** early in the chip design process, before the lengthy **placement and routing** process, could revolutionize the field; saving time, resources, and improving chip efficiency.
- We aim to enhance the **graph representation** of the chips to better predict congestion problem areas.



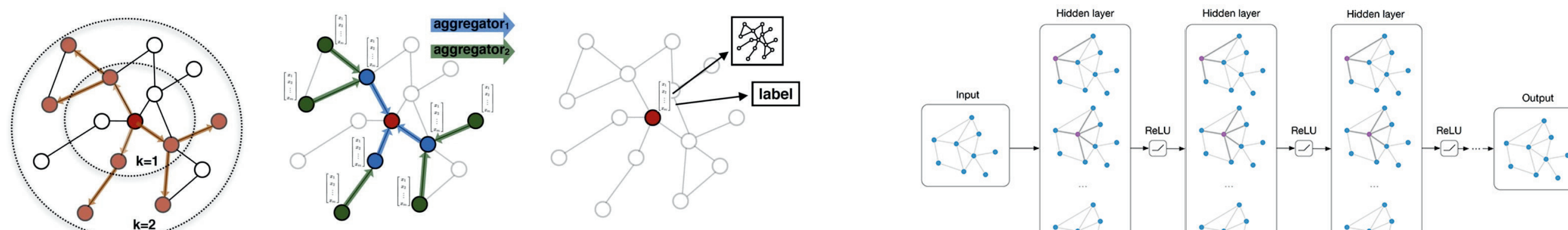
Data and Model Overview

Data Overview

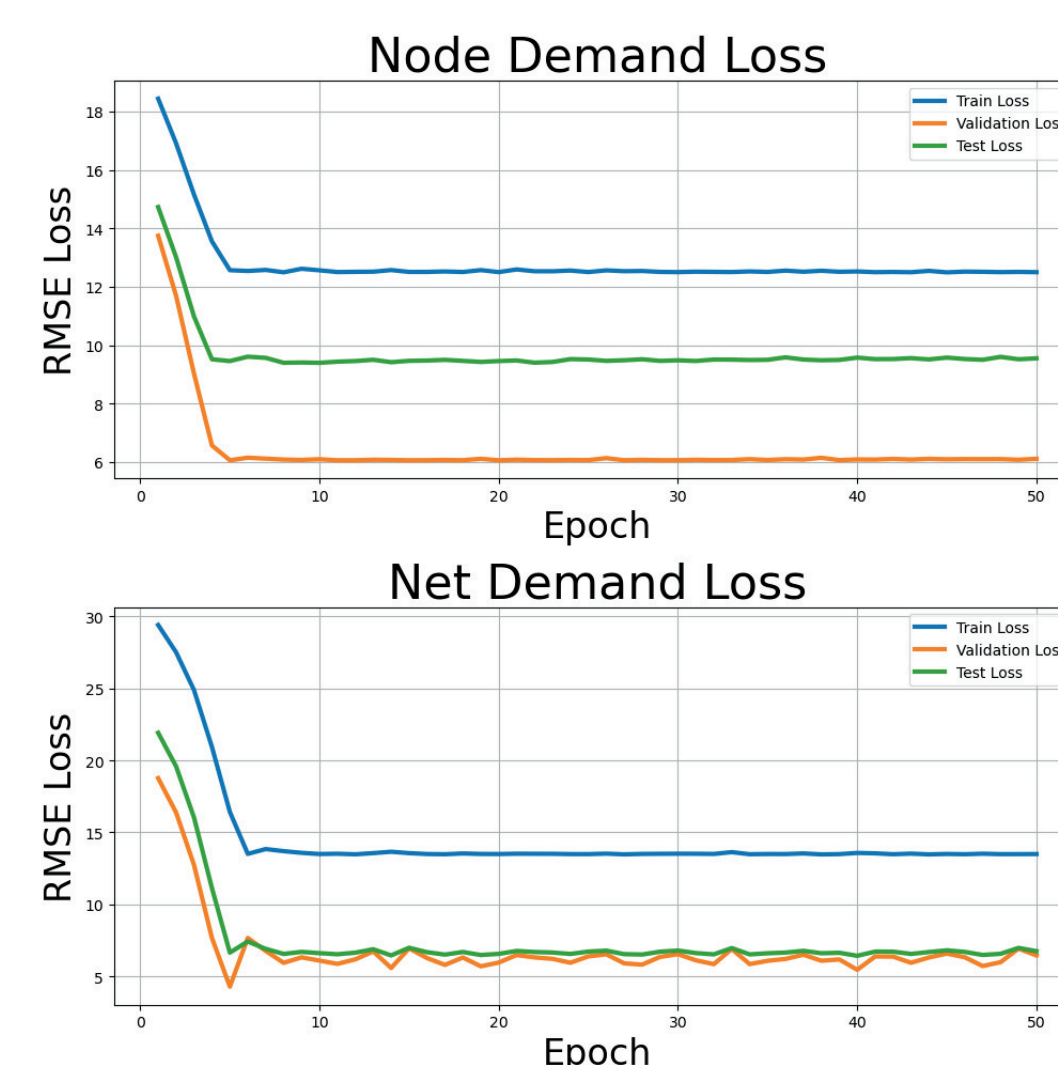
- Dataset comprises of **12 Superblue circuits** from (Viswanathan, 2012), with netlists (connectivity of an electronic circuit) containing 400K to 1.3M cells and nets.
- DE-HNN model processes netlists into **directed hypergraph representations**, with cells as nodes while nets are directed hyperedges.
- Node features include physical cell features (**size, cell type**) and topological features (**Laplacian eigenvectors, Persistence Diagrams, and neighborhood degree distrs.**).



Model Overview and Limitations



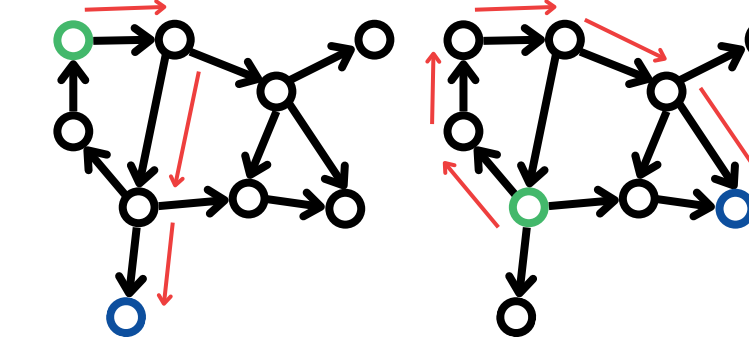
- The existing **DE-HNN model architecture** comprises of **3 Hypergraph Convolutional Layers** with **ReLU** activation functions to predict node demand.
- The DE-HNN graph representation uses **virtual nodes** to **aggregate** graph partitions generated by **METIS** to enhance long-range message passing, which is a common problem in GNNs.
- However, the current implementation has **limitations** in accurately capturing specific **long-range dependencies**, especially for nodes impacting congestion but are not well-represented within the existing partitions.
- Additionally, the existing model does not directly address the **gap** between the **topological** and **physical** representation.



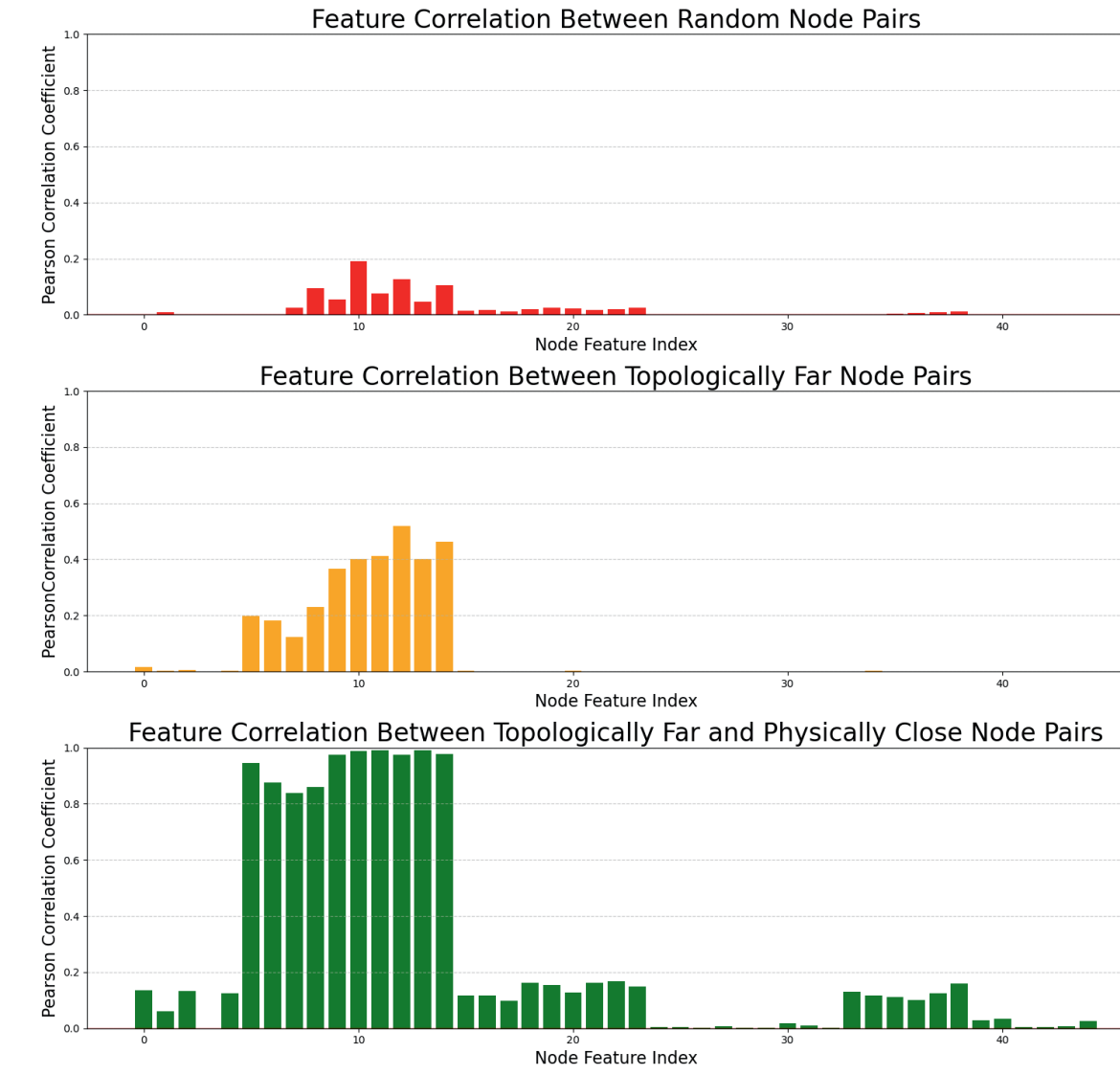
Methodology

Method 1: Adding Connections with Random Walks

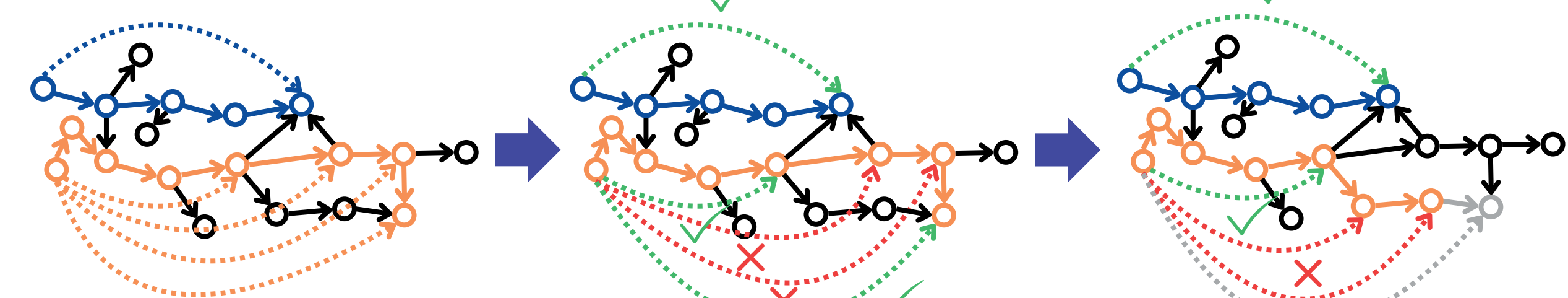
- Random walks** to enhance **long range communication** between important **topologically distant** node pairs.
- A direct way of facilitating message passing between far nodes is to **add direct connections** between them.
- Adding dummy **skip connections** between far source-destination pairs that contribute to **same congested area**.



Feature Correlation Analysis Across Different Node Pair Datasets



- Using **physical placement** data, constructing datasets of random node pairs, topologically far node pairs, and topologically far yet physically close node pairs (valid pairs).
- Found **high node feature correlation** in valid pairs for **in-flow persistence diagrams (PD)**.
- Built **XGBoost Classifier** as proxy for **physical proximity validation** during random walks.
- Analyzed paths of random walks to build **weighted random walk** algorithm using feature **entropy and stability**.
- Added **early stopping** to prevent **over-smoothing** of graph from excessive noise.



Random Walk Connections

XGBoost Validation

Validation w/ Early Stopping and Weighted Paths

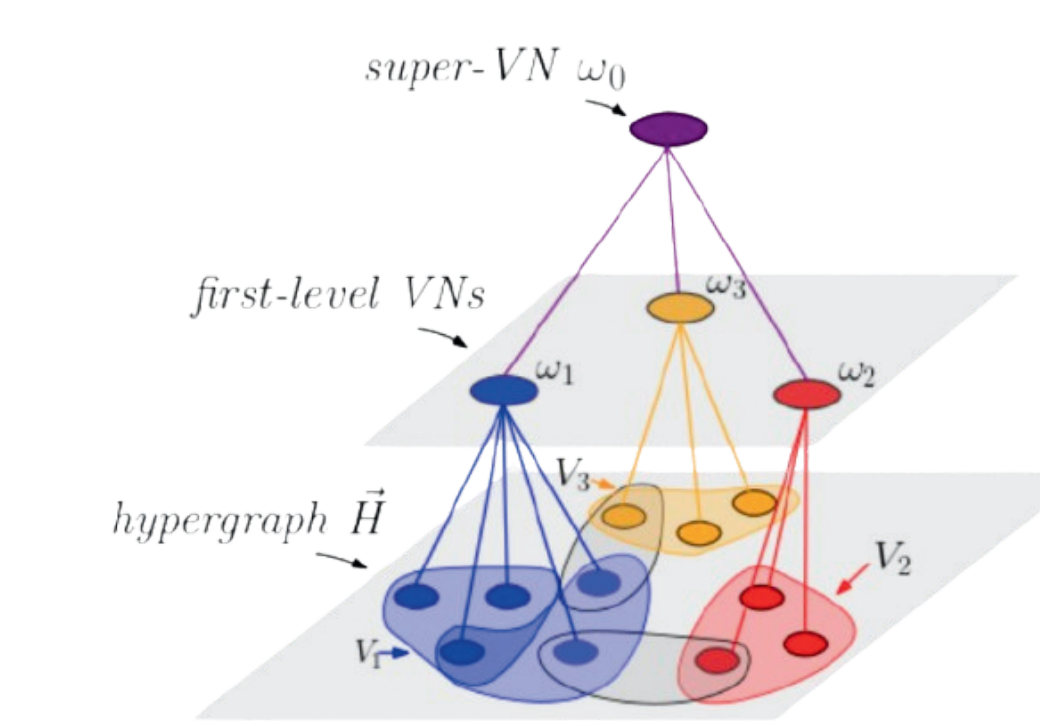
Method 2: Alternative Partitioning for Virtual Nodes

Louvain Partitioning

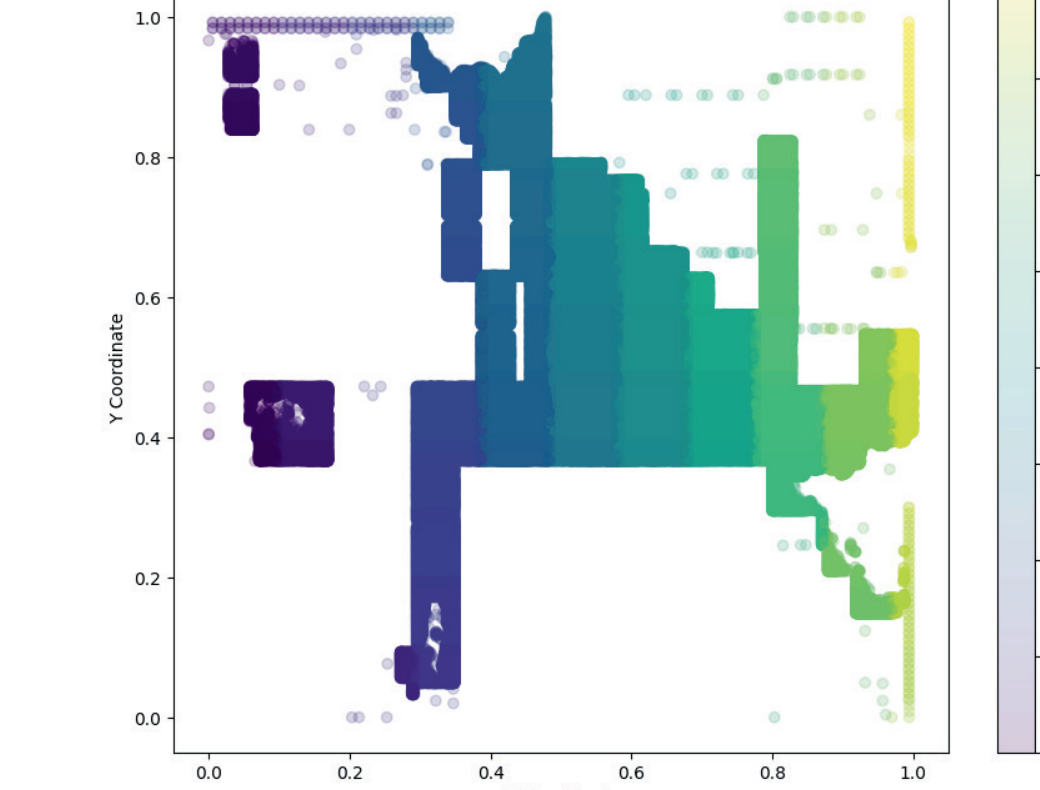
- We aim to **uncover meaningful substructures** within the chip designs using the **Louvain** algorithm for **community detection**.
- We find an average of **0.35 graph modularity** for the communities detected, showing **moderate separable community structure** identified from the graphs.
- We **partition nodes** into **communities** based on their modularity using the Louvain algorithm which replace the initial METIS partitions.

Physical Approximation

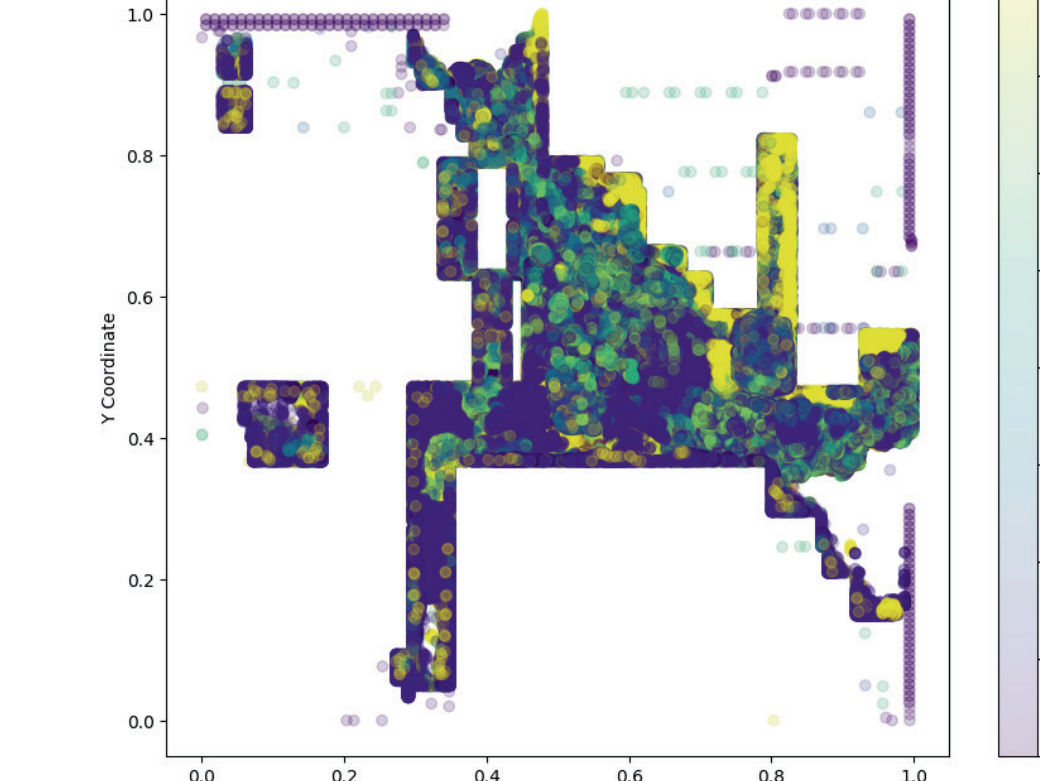
- DE-HNN was shown to perform **10% better** on MAE and RMSE using the **true physical placement** based partitioning and features
- We aim to **approximate** the true **physical placement partitions** using the **node features**
- Performed **feature analysis** on true physical partitions to find **features that correlate** to nodes being **physically close**.
- Partitioned** nodes based on these features using **two different clustering algorithms**
 - Standard K-Means**
 - Bisecting K-Means**



Node Locations Colored by True Placement Partition Group



Node Locations Colored by K-Means Partition Group



Results

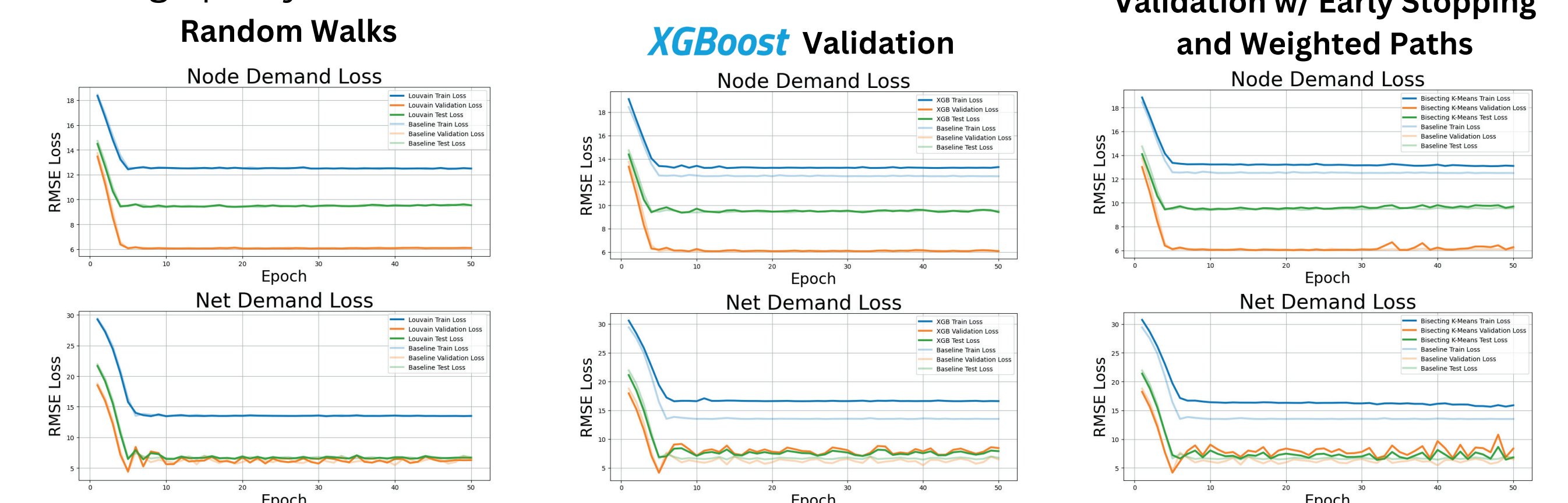
Method 1: Adding Connections with Random Walks

- Random Walk**: Performed worse for both node and net RMSE loss (+1.61%, +13.52%).
- XGBoost**: Marginal improvement for nodes (-0.64%), much worse for nets (+16.65%).
- Weighted Walks, Early Stopping**: Marginally better for nodes (-0.24%), significantly worse for nets (+15.70%).
- Results do **not yield much improvement**, possibly due to the overall graph structure being more meaningful.
- Improvement w/o early stopping points to adding edges.
- XGBoost Classifier** model shows promise (**93% accuracy**) in classifying nodes as physically close or not purely through proxy features.

XGBoost Performance

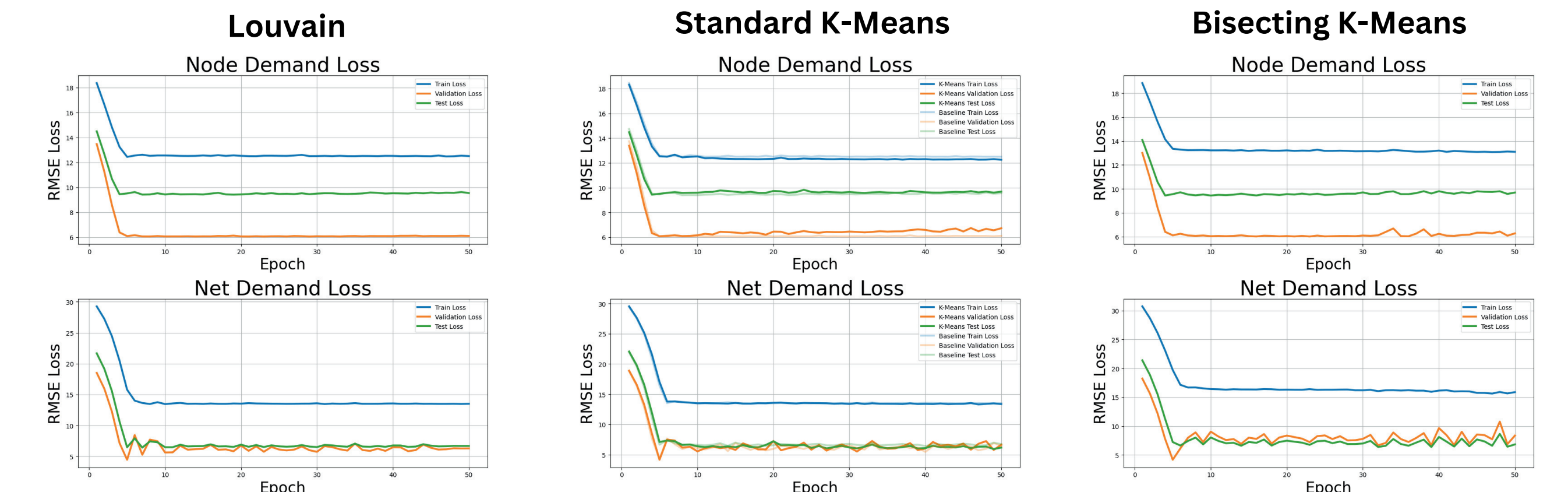
	precision	recall	f1-score	support
0	0.958	0.900	0.959	262053
1	0.625	0.613	0.619	27392
accuracy	0.926	0.926	0.926	1
macro avg	0.792	0.787	0.789	279445
weighted avg	0.925	0.926	0.926	279445

Validation w/ Early Stopping and Weighted Paths



Method 2: Alternative Partitioning for Virtual Nodes

- Louvain**: Marginal improvement in node loss (-0.25%), slightly worse for nets (+3.29%). However runs much faster, showing potential for modularity partitioning for chips.
- Standard K-Means**: Slightly worse for nodes (+1.79%), nets improved greatly (-6.68%).
- Bisecting K-Means**: Marginally worse for nodes (+0.37%), improvement for nets (-4.38%).
- Feature based partitioning methods show promise for improving net predictions. Bisecting K-Means balances the partition sizes smoothing the changes in loss.



Takeaways and Next Steps

- Regarding **random walks**, more connections are needed make a difference in the overall graph representation. With the **XGBoost** model classifying physically close nodes well, a more comprehensive connection adding algorithm can change the graph meaningfully.
- The **Louvain** algorithm presents an interesting alternative to METIS, as the performance is very comparable while **improving runtime** through larger, modular node partitions.
- K-Means** on **placement related features** shows promise in **capturing the true placement** information **without** having to perform **place and route** for true placement information.
- Overall, our experiments provide **insights** into **graph ML** methods for **analog chip design**.

Model	Node Loss	% Diff	Net Loss	% Diff
Baseline DE-HNN	9.533	---	6.573	---
Random Walks	9.687	1.6%	7.461	13.5%
XGBoost Validation	9.472	-0.6%	7.667	16.6%
Weighted Walk, Early Stopping	9.510	-0.2%	7.605	15.7%
Louvain Community Detection	9.509	-0.3%	6.790	3.3%
K-Means	9.704	1.8%	6.134	-6.7%
Bisecting K-Means	9.569	0.4%	6.285	-4.4%

Acknowledgements

- We would like to thank our wonderful Qualcomm mentors Lindsey Kostas, Alex Hagen, and Arnav Ballani for this wonderful opportunity to work with experienced industry engineers. Their guidance and insights helped us navigate this daunting project.